



**FACULTAD DE
INGENIERÍA**
UNIVERSIDAD DA VINCI
DE GUATEMALA

Universidad Da Vinci De Guatemala

Facultad de Ingeniería

Carrera: Ingeniería en Sistemas



**FACULTAD DE
INGENIERÍA**

UNIVERSIDAD DA VINCI
DE GUATEMALA

Consultoría de Arquitectura de Datos - Caso Apple Music

Elder Donaldo Salazar Garrido

Número de carné: 202305764

Curso: Bases de Datos 2

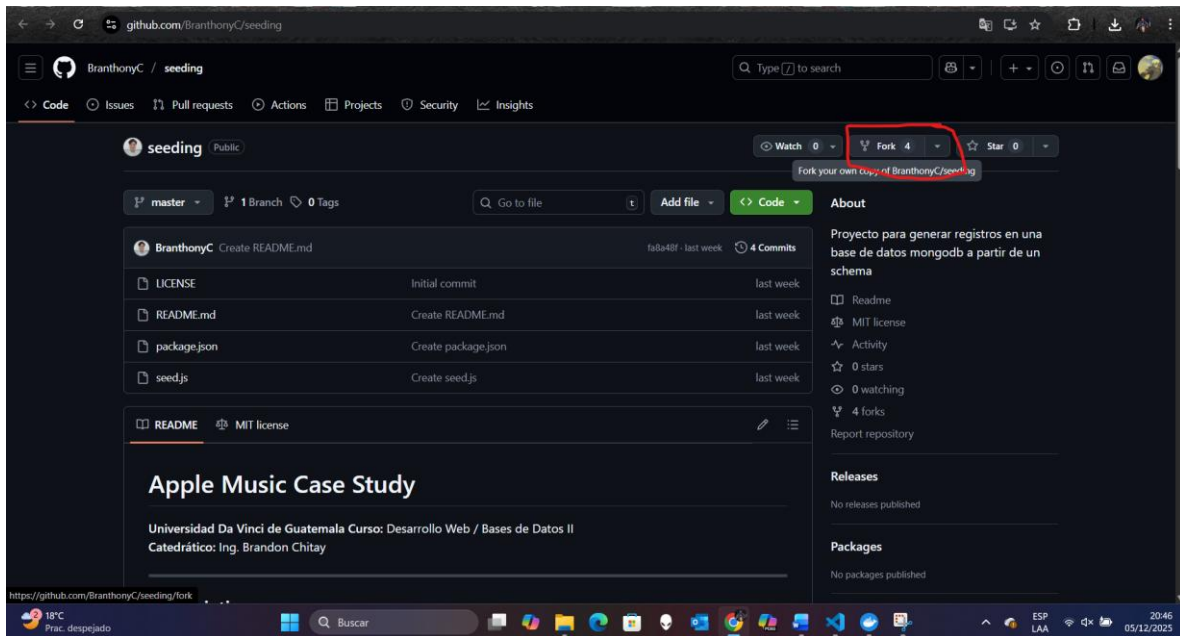
Guatemala, 05 de diciembre del 2025



FASE 1 — Setup:

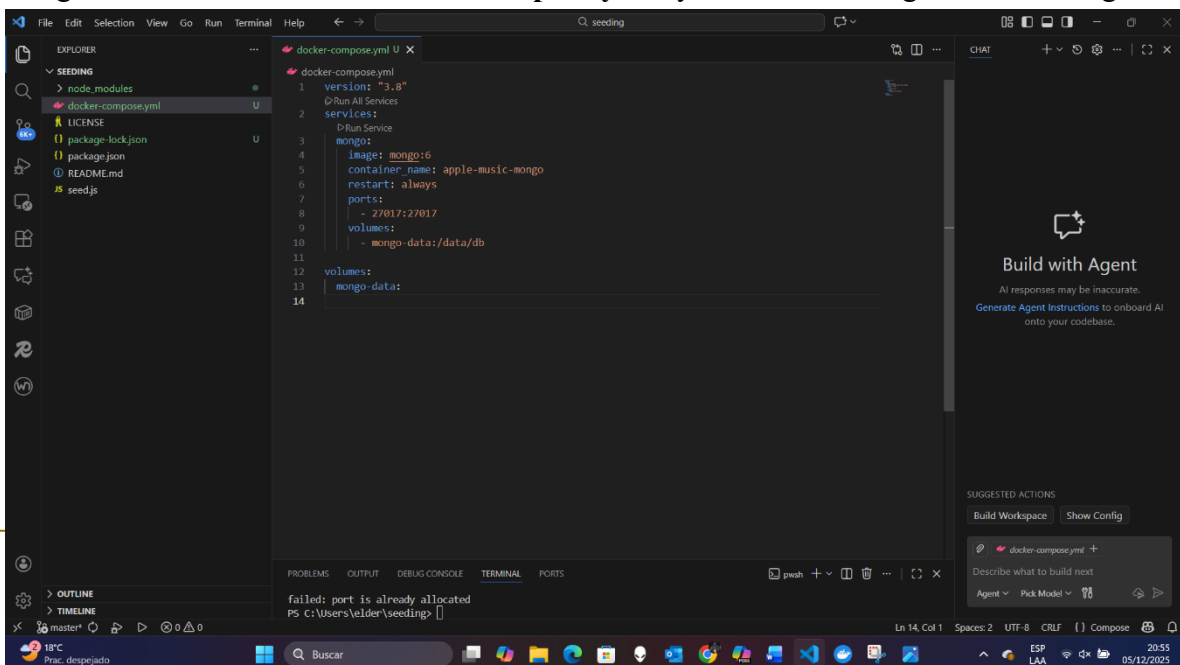
Hacemos Fork en el repositorio de <https://github.com/BranthonyC/seeding> y se nos creará una copia automáticamente en nuestra cuenta de GitHub.

Luego descargamos el repositorio que se nos creó con el Fork, ejecutamos el comando en la terminal y la ruta donde queremos descargarlo: en mi caso: `git clone https://github.com/Edonardo99/seeding` usuario de Github y luego nos movemos a la carpeta que hemos creado con el comando: `“cd seeding”`



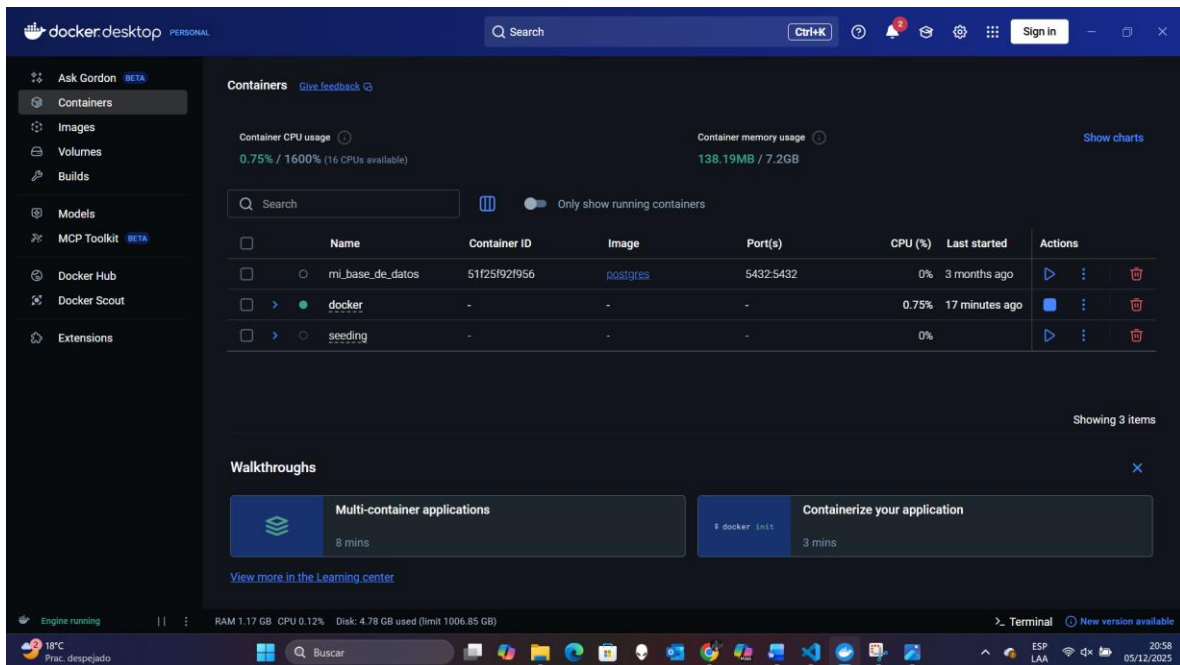
En la terminal luego que estamos en la carpeta **seeding**: ejecutamos el comando: `“npm install”`. El script de generación de datos utiliza Node.js. Instala las librerías necesarias:

Luego creamos el archivo: `“docker-compose.yml”` y estaremos configurando el código.

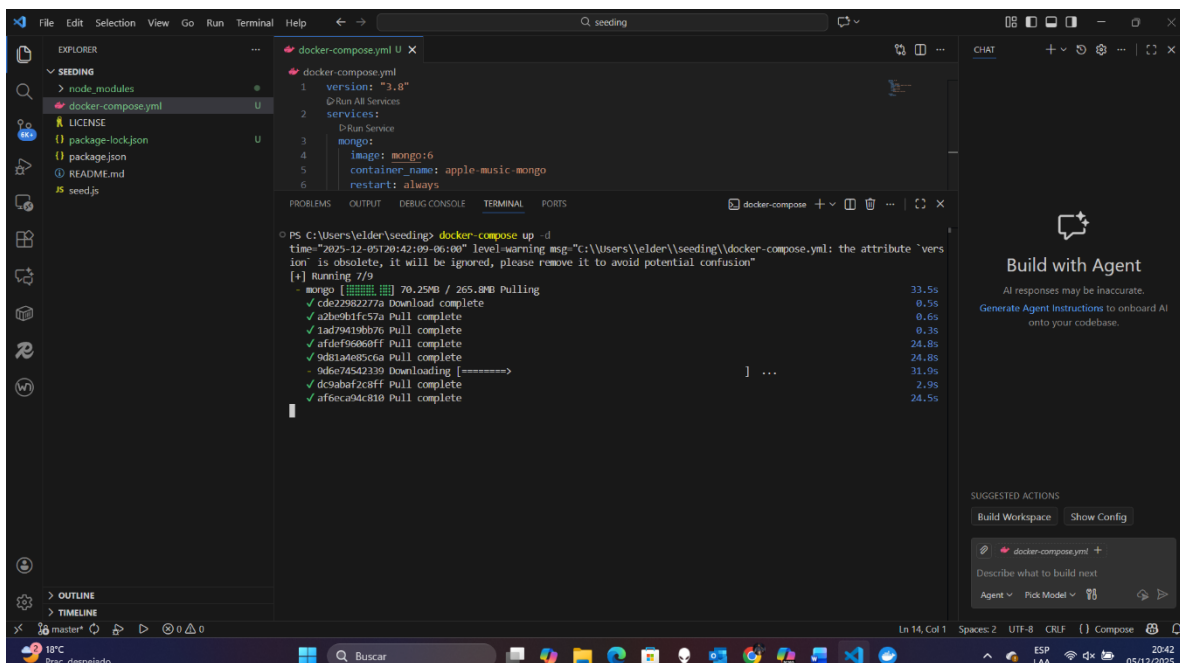




Procedemos a abrir el Docker desktop en nuestra PC antes de ejecutar el comando:
“**docker-compose up -d**”.



Ahora si. Usamos el comando: “**docker-compose up -d**” para levantar Infraestructura (Docker) si no abrimos el Docker desktop antes, este comando no funcionará. Ahora vemos que se están descargando algunos componentes para levantar la infraestructura.





En mi caso me tuve que mover a la carpeta raíz del proyecto: “C:\Users\elder\seeding>”, en la cual ejecuté el comando “**npm install**” para instalar las dependencias en la carpeta raíz y luego ejecuté el comando: “**node seed.js**”

```
PS C:\Users\elder\seeding> npm install
up to date, audited 14 packages in 1s

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\elder\seeding> node seed.js
Conectado a MongoDB. Limpiando colecciones viejas...
Generando Catálogo Musical...
Generando Usuarios...
Simulando Reproducciones (Streams)...
EXITO: Base de datos poblada.
- 100 Usuarios (20 son Zombies sin streams)
- 50 Canciones
- 5273 Streams generados

TIPS PARA EL ALUMNO:
1. Revisa los usuarios que NO están en la colección 'streams'.
2. Observa que 'songs' tiene el nombre del artista embebido.
PS C:\Users\elder\seeding>
```

Podemos ver claramente en la captura de pantalla que dice: " **EXITO:** Base de datos poblada", entonces estamos listos para comenzar.

Nota importante: “No use los comandos: “**docker-compose up -d**

” y “**npm start**” porque no me levantaba la base de datos, por eso en su lugar use los dos comandos mencionados antes.

FASE 2 — Base de Datos

verifica que ves la base (solo para confirmación visual)

Con los comandos:

1. `docker exec -it mi_mongodb mongosh`
2. `show dbs`
3. `use apple_music_db`
4. `show collections`

Obtenemos los siguientes resultados:



FACULTAD DE INGENIERÍA

UNIVERSIDAD DA VINCI
DE GUATEMALA

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\elder\seeding> node seed.js
PS C:\Users\elder\seeding> docker exec -it mi_mongodb mongosh
Current Mongosh Log ID: 6933a87b5b63c13605ce5f46
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.8
Using MongoDB:      8.0.14
Using Mongosh:       2.5.8

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2025-12-06T03:29:03.627+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2025-12-06T03:29:04.235+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
2025-12-06T03:29:04.236+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2025-12-06T03:29:04.236+00:00: For customers running the current memory allocator, we suggest changing the contents of the following sysfsFile
2025-12-06T03:29:04.236+00:00: We suggest setting the contents of sysfsFile to 0.
2025-12-06T03:29:04.236+00:00: vm.max_map_count is too low
2025-12-06T03:29:04.236+00:00: We suggest setting swappiness to 0 or 1, as swapping can cause performance problems.
-----

test> show dbs
...
admin          40.00 KiB
apple_music_db 360.00 KiB
config         72.00 KiB
local          76.00 KiB
test           152.00 KiB
test> use apple_music_db
switched to db apple_music_db
apple_music_db> show collections
artists
songs
streams
users
apple_music_db>
```



FASE 3 — Las 5 consultas

1) **Reporte de Regalías** — Total de segundos reproducidos por artista en los últimos 30 días

Comando para utilizar:

```
PS C:\Users\elder\seeding> docker exec -it mi_mongodb mongosh
artists
songs
streams
users
apple_music_db> db.streams.aggregate([
... {
...   $match: {
...     date: { $gte: new Date(date.now() - 30*24*60*60*1000) } // últimos 30 días
...   },
... },
... {
...   $group: {
...     _id: "$artist_id",
...     totalSeconds: { $sum: "$seconds_played" },
...     plays: { $sum: 1 }
...   },
... },
... // traer nombre del artista
... {
...   $lookup: {
...     from: "artists",
...     localField: "_id",
...     foreignField: "_id",
...     as: "artist"
...   },
... },
... { $unwind: "$artist" },
... {
...   $project: {
...     _id: 0,
...     artistId: "$_id",
...     artistName: "$artist.name",
...     totalSeconds: 1,
...     plays: 1
...   },
... },
... { $sort: { totalSeconds: -1 } }
... ])
[
```

Resultado:

```
totalSeconds: 1,
plays: 1
... }
... { $sort: { totalSeconds: -1 } }
... ])
[
  {
    totalSeconds: 185632,
    plays: 901,
    artistId: ObjectId('6933a5e2ee93a078dc8bcb51'),
    artistName: 'Bad Bunny'
  },
  {
    totalSeconds: 86224,
    plays: 436,
    artistId: ObjectId('6933a5e2ee93a078dc8bcb72'),
    artistName: 'Ricardo Arjona'
  },
  {
    totalSeconds: 77711,
    plays: 457,
    artistId: ObjectId('6933a5e2ee93a078dc8bcb5c'),
    artistName: 'Taylor Swift'
  },
  {
    totalSeconds: 76546,
    plays: 406,
    artistId: ObjectId('6933a5e2ee93a078dc8bcb7d'),
    artistName: 'Feld'
  },
  {
    totalSeconds: 74668,
    plays: 384,
    artistId: ObjectId('6933a5e2ee93a078dc8bcb67'),
    artistName: 'Metallica'
  }
]
apple_music_db>
```



2) Top 10 Regional (Guatemala) en últimos 7 días — canciones más escuchadas en GT

Comando para utilizar:

```
PS C:\Users\elder\seedings> docker exec -it mi_mongodb mongosh
apple_music_db> db.streams.aggregate([
... {
...   $match: {
...     date: { $gte: new Date(Date.now()) - 7*24*60*60*1000 }
...   },
... },
... // traer información del usuario para filtrar por país
... {
...   $lookup: {
...     from: "users",
...     localField: "user_id",
...     foreignField: "_id",
...     as: "user"
...   },
... },
... { $unwind: "$user" },
... { $match: { "user.country": "GT" } },
... // ahora agrupar por canción
... {
...   $group: {
...     _id: "$song_id",
...     plays: { $sum: 1 },
...     uniqueUsers: { $addToSet: "$user_id" } // opcional
...   },
... },
... // traer metadata de la canción (title, artist_name)
... {
...   $lookup: {
...     from: "songs",
...     localField: "_id",
...     foreignField: "_id",
...     as: "song"
...   },
... },
... { $unwind: "$song" },
... {
```

Resultado:

```
[
  {
    plays: 67,
    songId: ObjectId('6933a5e2ee93a078dc8bc52'),
    title: 'Let's Stay Together',
    artist: 'Bad Bunny',
    uniqueListeners: 36
  },
  {
    plays: 12,
    songId: ObjectId('6933a5e2ee93a078dc8bc56'),
    title: 'Frenesi',
    artist: 'Bad Bunny',
    uniqueListeners: 10
  },
  {
    plays: 11,
    songId: ObjectId('6933a5e2ee93a078dc8bc7c'),
    title: 'The Gypsy',
    artist: 'Ricardo Arjona',
    uniqueListeners: 10
  },
  {
    plays: 10,
    songId: ObjectId('6933a5e2ee93a078dc8bc66'),
    title: 'You Always Hurt the One You Love',
    artist: 'Taylor Swift',
    uniqueListeners: 10
  },
  {
    plays: 10,
    songId: ObjectId('6933a5e2ee93a078dc8bc5e'),
    title: 'No One',
    artist: 'Taylor Swift',
    uniqueListeners: 10
  },
  {
    plays: 9,
    songId: ObjectId('6933a5e2ee93a078dc8bc5f'),
    title: 'No Scrubs',
  }
]
```




3) Usuarios Zombis (Churn Risk) — Premium con cero reproducciones en 30 días

Comando para utilizar:

```
PS C:\Users\elder\seedings> docker exec -it mi_mongodb mongosh
songId: ObjectId('6933a5e2ee93a078dc8bcb6b'),
title: 'Hold On',
artist: 'Metallica',
uniqueListeners: 6
}
apple_music_db.users.aggregate([
... { $match: { subscription: "Premium" } },
... {
... $lookup: {
... from: "streams",
... let: { userId: "$_id" },
... pipeline: [
... { $match:
... { $expr:
... { $and: [
... { $eq: ["$user_id", "$userId" ] },
... { $gte: ["$date", new Date(Date.now() - 30*24*60*60*1000) ] }
... ]}
... }
... },
... as: "recentStreams"
... }
... },
... // quedamos solo con quienes no tienen streams recientes
... { $match: { recentStreams: { $size: 0 } } },
... {
... $project: {
... _id: 1,
... username: 1,
... email: 1,
... country: 1,
... subscription: 1,
... lastActive: { $ifNull: [ { $max: "$recentStreams.date" }, null ] } // null porque no tiene
... }
... }
... ]
}
```

Resultado:

```
PS C:\Users\elder\seedings> docker exec -it mi_mongodb mongosh
{
  _id: ObjectId('6933a5e2ee93a078dc8bcbdb'),
  username: 'Alyis.Koepp31',
  email: 'Alyis.Koepp31@hotmail.com',
  country: 'GT',
  subscription: 'Premium',
  lastActive: null
},
{
  _id: ObjectId('6933a5e2ee93a078dc8bcbdb'),
  username: 'Guiseppe.Orn',
  email: 'Guiseppe.Orn@gmail.com',
  country: 'GT',
  subscription: 'Premium',
  lastActive: null
},
{
  _id: ObjectId('6933a5e2ee93a078dc8bcbdb'),
  username: 'Tiffany.Cruickshank',
  email: 'Tiffany.Cruickshank@hotmail.com',
  country: 'GT',
  subscription: 'Premium',
  lastActive: null
},
{
  _id: ObjectId('6933a5e2ee93a078dc8bcbdb'),
  username: 'Jerrold.Schuster',
  email: 'Jerrold.Schuster@gmail.com',
  country: 'NI',
  subscription: 'Premium',
  lastActive: null
},
{
  _id: ObjectId('6933a5e2ee93a078dc8bcbdb'),
  username: 'Darian.Thiel',
  email: 'Darian.Thiel@gmail.com',
  country: 'PW',
  subscription: 'Premium',
  lastActive: null
},
}
```




4) Demografía por género (Reggaeton) — distribución por rangos de edad

Filtramos streams cuyas canciones sean género: "Reggaeton", vinculamos usuario y calculamos edad en años, luego bucket:

Comando para utilizar:

```
PS C:\Users\elder\seeding> docker exec -it mi_mongodb mongosh
apple_music_db> db.streams.aggregate([
  ... // unir canción para obtener género
  ... {
    $lookup: {
      from: "songs",
      localField: "song_id",
      foreignField: "_id",
      as: "song"
    }
  },
  ... { $unwind: "$song" },
  ... { $match: { "song.genre": "Reggaeton" } },
  ... // unir usuario para obtener birth_date
  ... {
    $lookup: {
      from: "users",
      localField: "user_id",
      foreignField: "_id",
      as: "user"
    }
  },
  ... { $unwind: "$user" },
  ... // calcular edad (aprox. en años)
  ... {
    $addFields: {
      age: {
        $floor: {
          $divide: [
            { $subtract: [new Date(), "$user.birth_date"] },
            1000 * 60 * 60 * 24 * 365
          ]
        }
      }
    }
  },
  ... // agrupar por usuario para evitar contar al mismo usuario varias veces
  ... { $group: { _id: "$user_id", count: { $sum: 1 } } }
])
```

Resultado:

```
PS C:\Users\elder\seeding> docker exec -it mi_mongodb mongosh
apple_music_db> db.streams.aggregate([
  ... { $group: { _id: "$user_id", count: { $sum: 1 } } },
  ... // agrupar por usuario para evitar contar al mismo usuario varias veces
  ... { $group: { _id: "$user_id", count: { $sum: 1 } } },
  ... // convertir a buckets/rangos
  ... {
    $bucket: {
      bucketBy: "$age",
      boundaries: [0, 15, 20, 30, 40, 50, 100],
      default: "100+",
      output: {
        count: { $sum: 1 }
      }
    }
  }
])
[
  { _id: 15, count: 13 },
  { _id: 20, count: 22 },
  { _id: 30, count: 19 },
  { _id: 40, count: 22 },
  { _id: 50, count: 4 }
]
```



5) Heavy Users (Bad Bunny) — 5 usuarios que más canciones distintas escucharon de Bad Bunny

Primero obtener el `_id` de Bad Bunny:

Comando para utilizar:

```
PS C:\Users\elder\seeding> docker exec -it mi_mongodb mongosh
...
objectId('6933a5e2ee93a078dc8bcb51')
apple_music_db.db.streams.aggregate([
  ...
  {
    $match: {
      ...
      artist_id: objectId('6933a5e2ee93a078dc8bcb51')
    },
    ...
  },
  {
    $group: {
      _id: '$user_id',
      uniqueSongs: { $addToSet: '$song_id' }
    },
    ...
  },
  {
    $project: {
      distinctSongsCount: { $size: '$uniqueSongs' }
    },
    ...
  },
  {
    $sort: { distinctSongsCount: -1 },
    $limit: 5,
    ...
  },
  {
    $lookup: {
      from: 'users',
      localField: '_id',
      foreignField: '_id',
      as: 'user'
    },
    ...
  },
  { $unwind: '$user' },
  ...
  {
    $project: {
      _id: 0,
      userId: '$_id',
      username: '$user.username',
      country: '$user.country',
      subscription: '$user.subscription',
      distinctSongsCount: 1
    }
  }
])
```

Resultado:

```
...
...
...
[
  {
    distinctSongsCount: 10,
    userId: objectId('6933a5e2ee93a078dc8bcb51'),
    username: 'Elroy28',
    country: 'GT',
    subscription: 'Premium'
  },
  {
    distinctSongsCount: 10,
    userId: objectId('6933a5e2ee93a078dc8bcb90'),
    username: 'Bridgette84',
    country: 'FR',
    subscription: 'Free'
  },
  {
    distinctSongsCount: 10,
    userId: objectId('6933a5e2ee93a078dc8bcb33'),
    username: 'Vinnie.Shields',
    country: 'GO',
    subscription: 'Premium'
  },
  {
    distinctSongsCount: 10,
    userId: objectId('6933a5e2ee93a078dc8bcb11'),
    username: 'Porter89',
    country: 'GT',
    subscription: 'Free'
  },
  {
    distinctSongsCount: 10,
    userId: objectId('6933a5e2ee93a078dc8bcbaf'),
    username: 'Neil.Nacekovic76',
    country: 'GT',
    subscription: 'Premium'
  }
]
apple_music_db
```



FASE 4 — Dashboard Profesional:

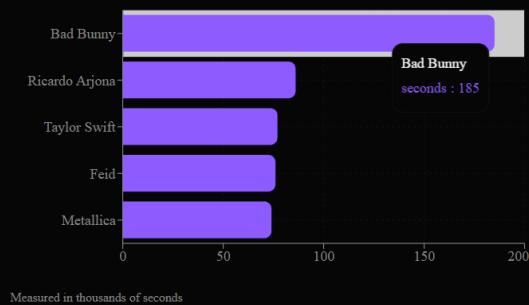
Implementación de Dashboard con Asistencia de IA

Para la visualización y análisis de los resultados de las cinco consultas principales, se utilizó la herramienta de inteligencia artificial **v0.app**. Esta plataforma de diseño asistido por IA fue empleada para **generar de manera rápida y eficiente el código fuente y el diseño del dashboard**. El uso de v0.app permitió transformar los datos obtenidos de las consultas en una interfaz gráfica funcional y estéticamente agradable, lo que aceleró significativamente el proceso de desarrollo y permitió enfocarnos en la lógica de las consultas en lugar de la maquetación front-end.

Apple Music Admin

Analytics and insights dashboard

Top Artists - Last 30 days



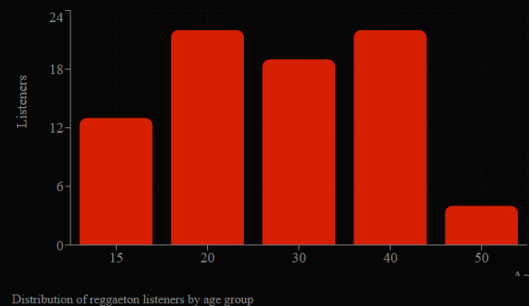
Top 10 Songs Guatemala - Last 7 days

Title	Artist	Plays	Listeners
Ella Baila Sola	Eslabon Armado	12.4k	8.2k
Un x100to	Grupo Frontera	10.8k	7.1k
La Jumpa	Bad Bunny	9.3k	6.5k
Ella y Yo	Aventura	8.7k	5.9k
Despecha	Rosalía	8.1k	5.4k
Ojitos Lindos	Bad Bunny	7.6k	5.1k
La Jumpa	Arcángel	7.2k	4.8k
Ella Baila Sola	Peso Pluma	6.9k	4.5k
Titi Me Preguntó	Bad Bunny	6.4k	4.2k
Un x100to	Grupo Sombra	5.8k	3.9k

Premium Zombies - Last 30 days

Total Premium Zombies		
16		
Active Accounts		
Username	Country	Email
user_5847	Guatemala	user5847@music.com
zombie_listener	Guatemala	zombie@music.com
night_owl_92	Guatemala	nightowl92@music.com
premium_ghost	Guatemala	premiumghost@music.com

Reggaeton Demographics



Heavy Users - Bad Bunny

Username	Country	Subscription	Distinct Songs
bunny_fan_001	Guatemala	Premium Plus	847
reggaeton_lover	Mexico	Premium	723
music_junkie	Guatemala	Premium Plus	1204
daily_playlist	El Salvador	Premium	654
night_listener	Honduras	Premium Plus	892
artist_explorer	Guatemala	Premium	1456
top_fan	Nicaragua	Premium Plus	678
sound_seeker	Guatemala	Premium	945



Consulta 1 — Top Artistas por Regalías (30 días)

Endpoint: [GET /api/charts/top-artists?days=30](#)

Devuelve el ranking de artistas con mayor tiempo total reproducido en los últimos 30 días, calculado por segundos escuchados. Esta información se usa para estimar pagos de regalía

```
{
  "generated_at": "2025-12-06T00:00:00Z",
  "days": 30,
  "data": [
    {
      "artistName": "Bad Bunny",
      "artistId": "6933a5e2ee93a078dc8bcb51",
      "plays": 901,
      "totalSeconds": 185632
    },
    {
      "artistName": "Ricardo Arjona",
      "artistId": "6933a5e2ee93a078dc8bcb72",
      "plays": 436,
      "totalSeconds": 86224
    },
    {
      "artistName": "Taylor Swift",
      "artistId": "6933a5e2ee93a078dc8bcb5c",
      "plays": 457,
      "totalSeconds": 77711
    },
    {
      "artistName": "Feid",
      "artistId": "6933a5e2ee93a078dc8bcb7d",
      "plays": 426,
      "totalSeconds": 76546
    },
    {
      "artistName": "Metallica",
      "artistId": "6933a5e2ee93a078dc8bcb67",
      "plays": 384,
      "totalSeconds": 74668
    }
  ]
}
```



Consulta 2 — Top Canciones por Región (7 días)

Endpoint: <GET /api/charts/top-songs?region=GT&days=7>

Retorna el Top 10 de canciones más escuchadas en un país específico durante los últimos 7 días.

```
{
  "generated_at": "2025-12-06T00:00:00Z",
  "region": "GT",
  "days": 7,
  "data": [
    {
      "rank": 1,
      "title": "Let's Stay Together",
      "artist": "Bad Bunny",
      "plays": 67,
      "uniqueListeners": 36
    },
    {
      "rank": 2,
      "title": "Frenesi",
      "artist": "Bad Bunny",
      "plays": 12,
      "uniqueListeners": 10
    },
    {
      "rank": 3,
      "title": "The Gypsy",
      "artist": "Ricardo Arjona",
      "plays": 11,
      "uniqueListeners": 10
    },
    {
      "rank": 4,
      "title": "You Always Hurt the One You Love",
      "artist": "Taylor Swift",
      "plays": 10,
      "uniqueListeners": 10
    }
  ]
}
```



Consulta 3 — Usuarios Premium que No Escucharon en 30 días (Zombies)

Endpoint: [GET /api/users/zombies?subscription=Premium&days=30](#)

Devuelve usuarios Premium que no registran reproducciones en los últimos 30 días. Sirve para identificar usuarios inactivos (retención).

```
{
  "generated_at": "2025-12-06T00:00:00Z",
  "subscription": "Premium",
  "days": 30,
  "data": [
    {
      "userId": "6933a5e2ee93a078dc8bcbd8",
      "username": "Alvis.Koepp31",
      "email": "Abby.Pfannerstill@hotmail.com",
      "country": "GT",
      "lastActive": null
    },
    {
      "userId": "6933a5e2ee93a078dc8bcbdd",
      "username": "Berry.Hackett",
      "email": "Hoyt.Leannon31@yahoo.com",
      "country": "GT",
      "lastActive": null
    }
  ]
}
```



Consulta 4 — Demografía de oyentes de Reggaeton

Endpoint: [GET /api/analytics/reggaeton-demographics](#)

Entrega estadísticas por rangos de edad de oyentes que escuchan canciones de género Reggaeton. Ideal para marketing segmentado.

```
{
  "generated_at": "2025-12-06T00:00:00Z",
  "genre": "Reggaeton",
  "segments": [
    { "ageRange": "0-14", "listeners": 13 },
    { "ageRange": "15-19", "listeners": 22 },
    { "ageRange": "20-29", "listeners": 19 },
    { "ageRange": "30-39", "listeners": 22 },
    { "ageRange": "40-49", "listeners": 4 }
  ]
}
```




Endpoint: [GET /api/charts/heavy-listeners?artist=Bad Bunny&limit=5](#)

Devuelve los usuarios que han escuchado la mayor cantidad de canciones distintas de un artista específico.

```
{
  "generated_at": "2025-12-06T00:00:00Z",
  "artist": "Bad Bunny",
  "limit": 5,
  "data": [
    {
      "userId": "6933a5e2ee93a078dc8bcba5",
      "username": "Elroy28",
      "country": "GT",
      "subscription": "Premium",
      "distinctSongsCount": 10
    },
    {
      "userId": "6933a5e2ee93a078dc8bcb90",
      "username": "Bridgette84",
      "country": "TF",
      "subscription": "Free",
      "distinctSongsCount": 10
    }
  ]
}
```



Conclusiones

En este trabajo implementé una arquitectura de datos pensada para analizar el comportamiento de uso en un servicio similar a Apple Music. Decidí utilizar MongoDB porque el modelo documental permite guardar la información de forma más flexible y sin depender de joins complejos, especialmente al trabajar con grandes volúmenes de reproducciones. También utilicé el patrón de referencia extendida para tener datos clave del artista directamente en cada canción, lo cual facilita las consultas analíticas.

La semilla de datos que desarrollé genera un escenario realista: creé artistas conocidos, canciones, usuarios con diferentes países y edades, y reproducciones con un comportamiento similar al de una plataforma real. Esto me permitió obtener patrones interesantes, por ejemplo: el Top de artistas en los últimos días, las canciones más escuchadas por región (GT), la distribución por edades dentro del género reggaetón, los usuarios “zombis” que no han usado la app, y los heavy users de Bad Bunny.

Además, diseñé un dashboard prototipo en v0.dev para visualizar los resultados de las consultas. Aunque no está conectado directamente a la base de datos, el objetivo era mostrar cómo se verían los datos procesados: gráficas de barras, tablas, rankings y distribución por edades. Este prototipo serviría para que un analista pueda interpretar la información de forma rápida.

Finalmente, documenté los endpoints de la API que alimentarían el dashboard en un ambiente real. Esto define el “contrato” de datos, es decir, cómo cualquier aplicación externa podría consultar los resultados y mostrarlos. Esta parte es importante porque convierte el análisis en un servicio que puede utilizarse en otros sistemas.

En general, este proyecto me ayudó a integrar varios conceptos: diseño de base de datos documental, generación de datos para pruebas, consultas analíticas usando agregaciones, visualización de resultados y documentación de endpoints. Con esto, creo que logré una solución completa y clara, enfocada en análisis de datos reales.